



Acquisition and Analysis of Neural Data

Bernstein Center for Computational Neuroscience Berlin

- Lecturer: *Prof. Dr. Richard Kempter*
- Tutors: *Stefano Masserini, Ikhwan Bin Khalid*
- Problem Set 1 Deadline: *Tuesday 21st April, 2026 at 23:55*

Students:

1. Shokoofeh Hosseini
2. Anina Morgner

AI has not been used to solve the problems.

✓ 1. Firing rate estimation and convolution (10 points)

```
import numpy as np
import matplotlib.pyplot as plt

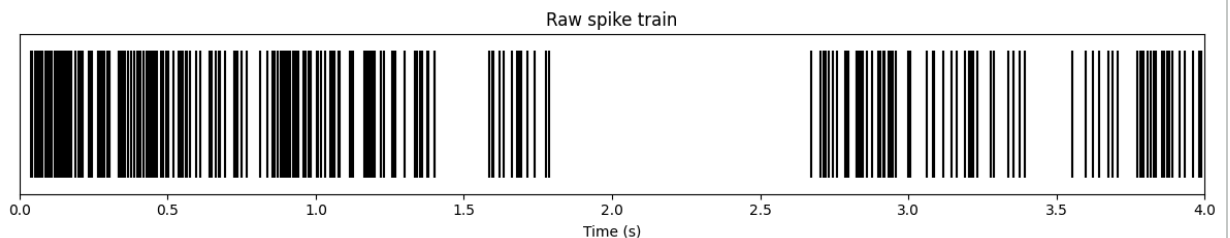
# =====
# Load data
# =====
def load_spike_times(filename):
    spike_times_ms = np.loadtxt(filename)
    spike_times_s = spike_times_ms / 1000.0
    return spike_times_s

spike_times_s = load_spike_times("/content/ExampleSpikeTimes1.dat")
```

(1 point) Plot the raw spike train.

```
# =====
# (a) Raw spike train
# =====
def plot_spike_train(spike_times_s, T, dt):
    plt.figure(figsize=(12, 2.5))
    plt.eventplot(spike_times_s, colors='black', lineoffsets=1, linelengths=0.8)
    plt.xlim(0, T)
    plt.ylim(0.5, 1.5)
    plt.yticks([])
    plt.xlabel("Time (s)")
    plt.title("Raw spike train")
    plt.tight_layout()
    plt.show()

T = 4.0                # trial length in s
dt = 0.0001            # temporal resolution = 0.1 ms
fs = 1 / dt
plot_spike_train(spike_times_s, T, dt)
```



Interpretation:

The train is not uniform over time. The temporal structure changes

Early segment ($\approx 0-1.5$ s): very dense $\rightarrow$ high activity.

Middle ($\approx 1.5-2.5$ s): almost silent $\rightarrow$ low activity.

Later ($\approx 2.5-4$ s): activity resumes, somewhat bursty.

(3 points) Construct spike-count histograms with non-overlapping bins of widths $\Delta t = 20, 50$ and 150 ms, and normalize them to obtain a firing rate in spikes/s.

```
# =====
# (b) Histogram rate estimates
```

```

# count number of spike: For a bin width  $\Delta t$ ,

# rate in bin= spike count in bin/  $\Delta t$ 

# =====

def plot_rate_histogram(spike_times_s, bin_widths_ms, T, dt):

    fig, axes = plt.subplots(len(bin_widths_ms), 1, figsize=(12, 8), sharex=True)

    for ax, bw_ms in zip(axes, bin_widths_ms):
        bw_s = bw_ms / 1000.0
        edges = np.arange(0, T + bw_s, bw_s)
        counts, _ = np.histogram(spike_times_s, bins=edges)
        rate = counts / bw_s # spikes/s
        centers = edges[:-1] + bw_s / 2

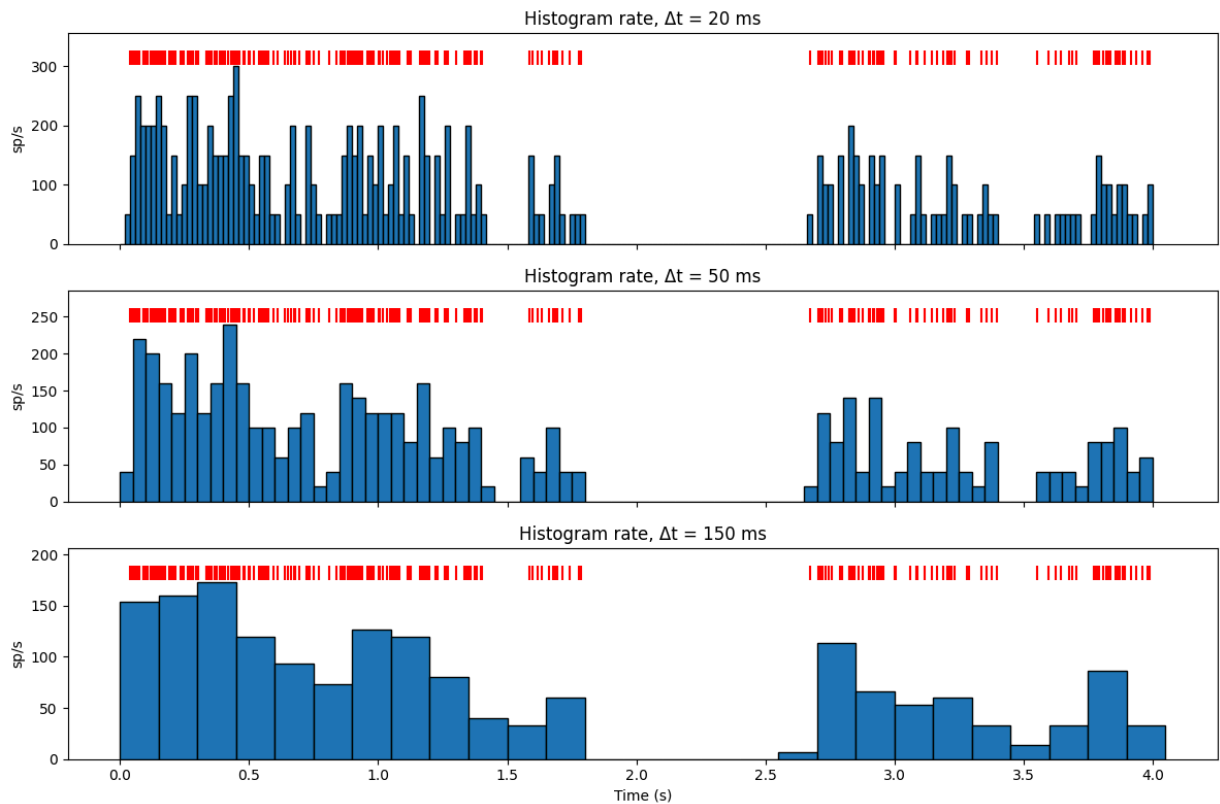
        ax.bar(centers, rate, width=bw_s, align='center', edgecolor='black')
        ax.eventplot(spike_times_s, colors='red',
                     lineoffsets=np.max(rate) * 1.05 if len(rate) > 0 else 1,
                     linelengths=np.max(rate) * 0.08 if len(rate) > 0 else 0.2)
        ax.set_ylabel("sp/s")
        ax.set_title(f"Histogram rate,  $\Delta t = \{bw\_ms\}$  ms")

    axes[-1].set_xlabel("Time (s)")
    plt.tight_layout()
    plt.show()

    return rate

bin_widths_ms = [20, 50, 150]
rate = plot_rate_histogram(spike_times_s, bin_widths_ms, T, dt)

```



Interpretation:

- 20 ms: high temporal detail, noisy estimate,
- 50 ms: smoother, still responsive,
- 150 ms: very smooth, but loses fine temporal structure.

In general, smaller bins mean high variance and low bias, and bigger bins the other way around.

```
time = np.arange(0, T, dt)
n_time = len(time)

# Discrete spike train rho(t)
spike_train = np.zeros(n_time)
spike_indices = np.round(spike_times_s / dt).astype(int)

spike_indices = spike_indices[(spike_indices >= 0) & (spike_indices < n_time)]
spike_train[spike_indices] = 1.0 / dt
```

```

# =====
# convolution-based estimate
# =====
def rate_from_kernel(spike_train, kernel, dt):
    """
    spike_train: discrete approximation to rho(t), in spikes/s
    kernel: discrete kernel samples, should approximate unit area
    dt: time step in s
    """
    rate = np.convolve(spike_train, kernel, mode='same') * dt
    return rate

def plot_firing_rate_with_kernel(kernel_name, bin_widths_ms, T, dt):
    fig, axes = plt.subplots(3, 1, figsize=(12, 8), sharex=True)
    for ax, bw_ms in zip(axes, bin_widths_ms):
        width_s = bw_ms / 1000.0

        match kernel_name:
            case 'rectangular':
                kernel = rectangular_kernel(width_s, dt)
            case 'gaussian':
                kernel = gaussian_kernel(width_s, dt)
            case 'alpha':
                kernel = alpha_kernel(width_s, dt)

        rate = rate_from_kernel(spike_train, kernel, dt)

        ax.plot(time, rate)
        ax.eventplot(spike_times_s, colors='red',
                     lineoffsets=np.max(rate) * 1.05,
                     linelengths=np.max(rate) * 0.08)
        ax.set_ylabel("sp/s")
        ax.set_title(f"{kernel_name}, Δt = {bw_ms} ms")

    axes[-1].set_xlabel("Time (s)")
    plt.tight_layout()
    plt.show()

```

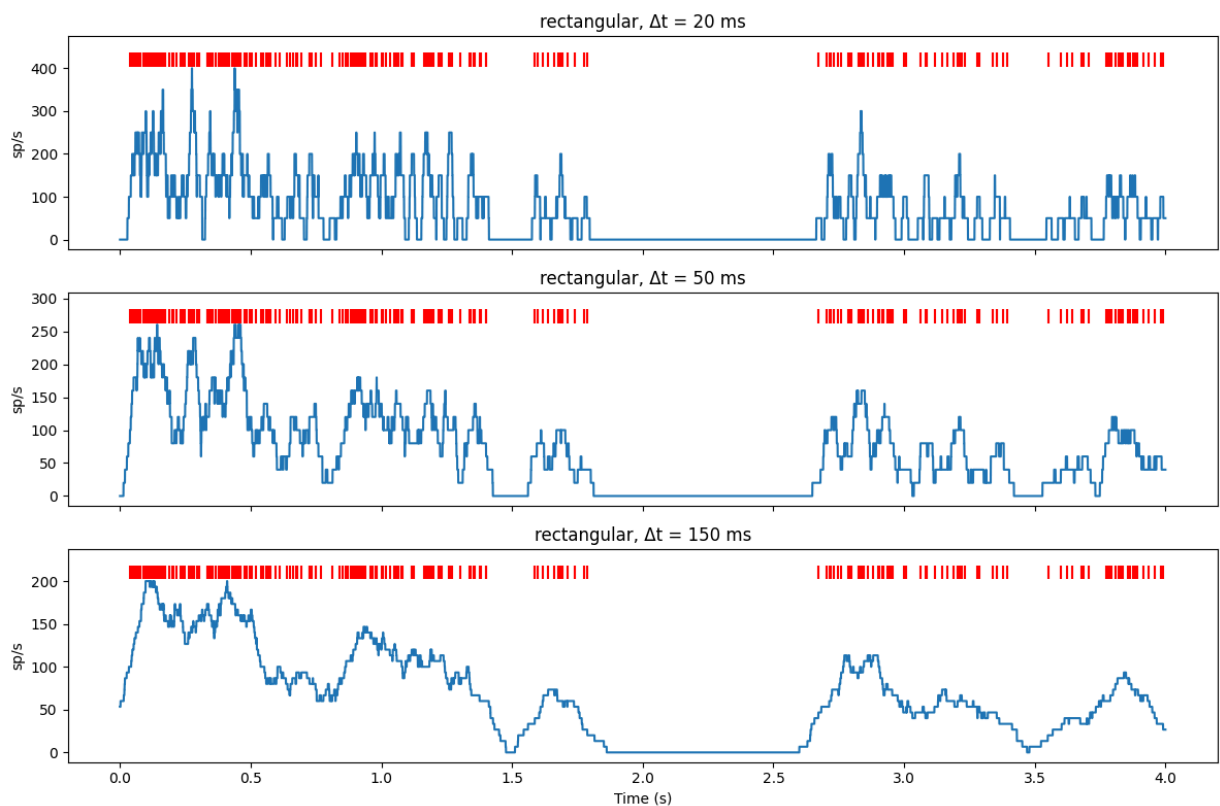
Differences convolution modes

- *full* returns the complete convolution (also edge cases where kernel center sits not on a data index), larger than the input signal
- *same* returns an array of the same size as the input, centered window corresponding to each time index; *same* includes one value per input timepoint, where the kernel is centered at that timepoint — even if part of the kernel lies outside the signal

For both, all values outside input sequence are assumed as zero.

```
# =====
# (c.i) Rectangular window
#  $w(t) = 1/\Delta t$  for  $|t| < \Delta t/2$ , else 0
# non-causal, centered
# =====
def rectangular_kernel(width_s, dt):
    n = int(np.round(width_s / dt))
    if n < 1:
        n = 1
    kernel = np.ones(n) / width_s
    # area approx = sum(kernel)*dt  $\approx 1$ 
    return kernel

plot_firing_rate_with_kernel('rectangular',[20, 50, 150], T, dt)
```



Interpretation:

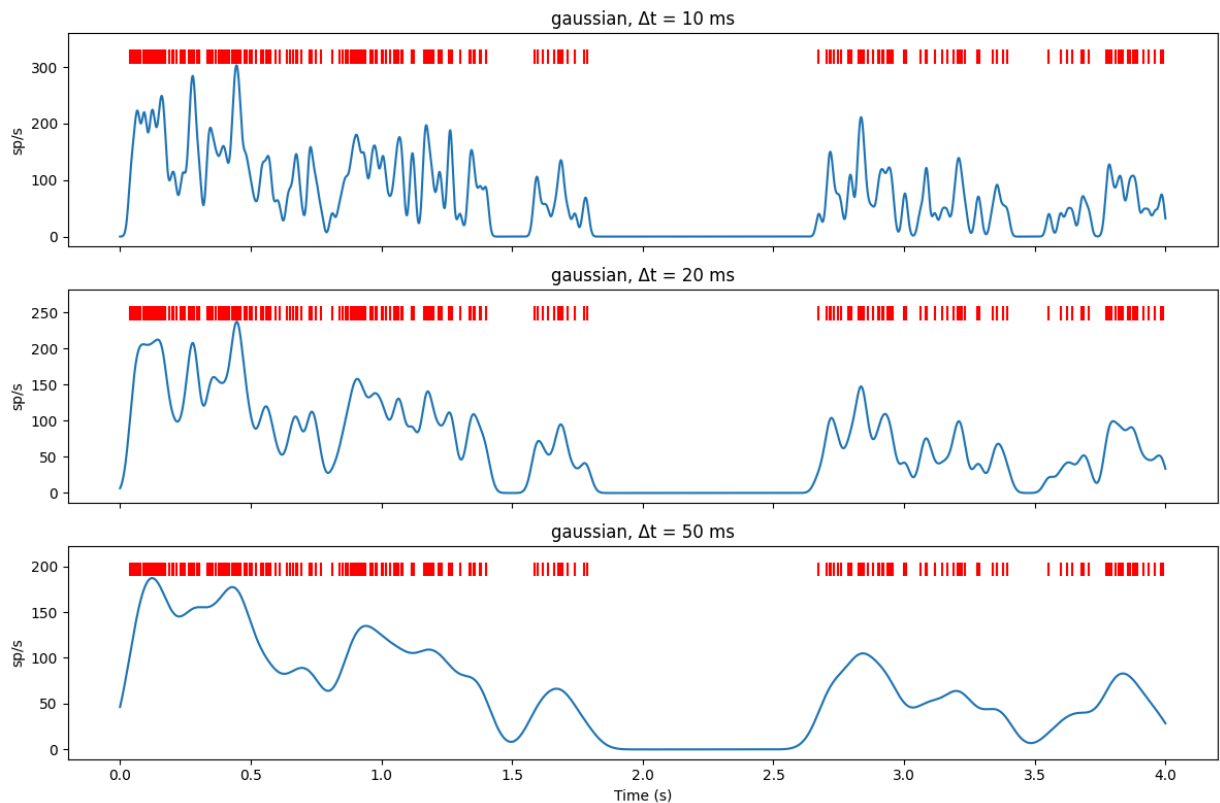
The kernel's width influences the smoothness of firing rate. The smaller width is noisy while the wider kernel's width produce smoother firing rate; however, the loses should

not be overlooked. In contrast to histogram, rates have better temporal resolutions and the placement of bin is not arbitrary.

Rectangular filter produces piecewise constant smoothing equivalent to a moving average. Rates are non-causal meaning the kernel is centered at t_i , so part of kernel lies before the spike and part lies after it. That means the estimated rate at time t can be influenced by a spike that has not yet happened.

```
# =====
# (c.ii) Gaussian window
#  $w(t) = 1/(\sqrt{2\pi}\sigma) \exp(-t^2 / (2\sigma^2))$ 
# non-causal, centered
# =====
def gaussian_kernel(sigma_s, dt, truncate=4):
    half_width = int(np.ceil(truncate * sigma_s / dt))
    tau = np.arange(-half_width, half_width + 1) * dt
    kernel = (1 / (np.sqrt(2 * np.pi) * sigma_s)) * np.exp(-(tau**2) / (2 * sig
    return kernel
```

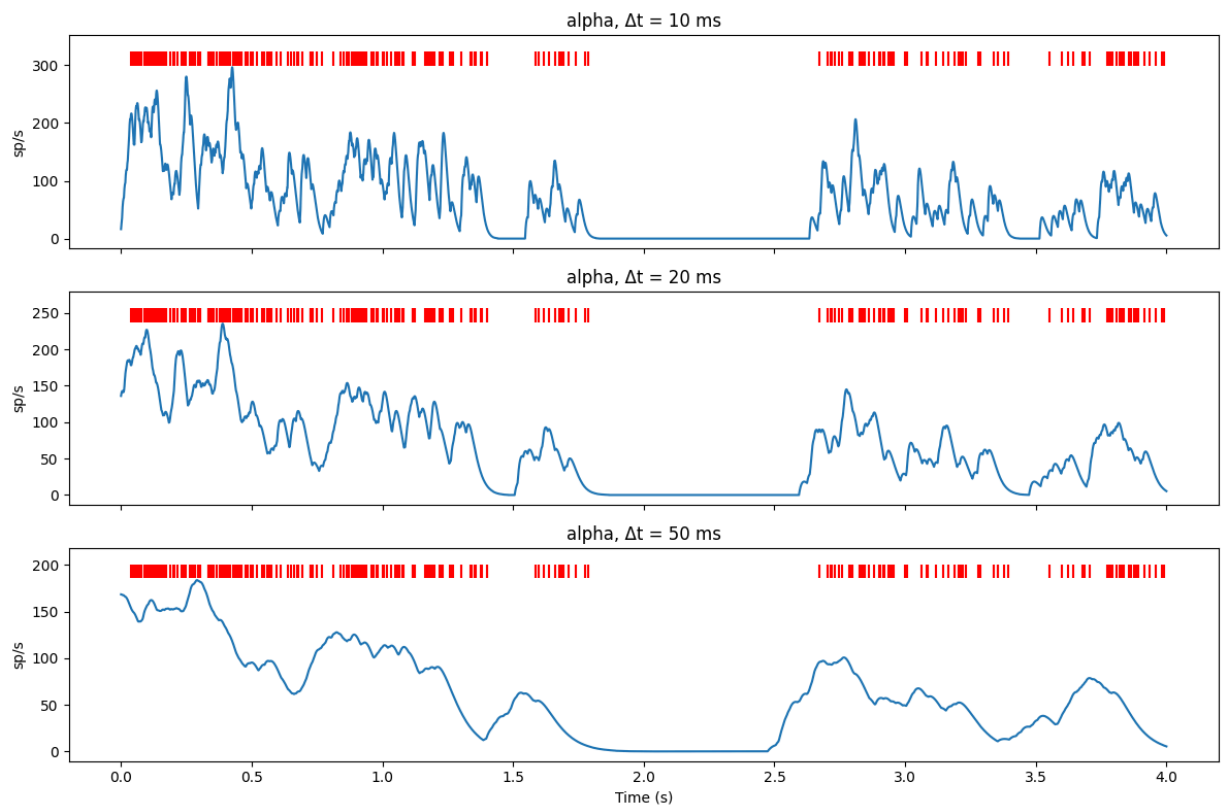
```
plot_firing_rate_with_kernel('gaussian',[10, 20, 50], T, dt)
```



Interpretation:

The Gaussian kernel produce smooth, infinitely differentiable estimates and avoids sharp discontinuities introduced by rectangular kernels. Nevertheless, it is still non-causal and correlated between different times.

```
# =====  
# (c.iii) Alpha window  
#  $w(t) = \alpha^2 t \exp(-\alpha t)$  for  $t \geq 0$ , else 0  
# causal  
# =====  
def alpha_kernel(tau_s, dt, truncate=8):  
    # tau_s = 1/alpha  
    alpha = 1.0 / tau_s  
    tmax = truncate * tau_s  
    tau = np.arange(0, tmax + dt, dt)  
    kernel = (alpha**2) * tau * np.exp(-alpha * tau)  
    return kernel  
plot_firing_rate_with_kernel('alpha',[10, 20, 50], T, dt)
```

Interpretation:

The alpha kernel is causal, which is an advantage in comparison to the rectangular and gaussian filter. The kernel is asymmetric, biased towards positive. There is no effect before the spike but it rises and then decays after the spike. Alpha filter is more biologically plausible but it is biased to positive times.

Why would one want to use a causal rather than a non-causal filter?

A causal filter does not use future spikes to estimate the rate at time t , so it ensures that the estimate at time t depends only on spikes at times $t_i \leq t$. Therefore it is appropriate for real-time systems and biologically plausible modeling, where only past and present information can influence the current estimate. A non-causal filter incorporates future spikes ($t_i < t$ and $t_i > t$), it is fine for offline analysis. It is important to keep in mind, that it artificially lets spikes influence the estimated rate before they occur (violate temporal realism). Nevertheless it improves smoothness.

```
# =====
# (d) Mean spike count-rate
# =====

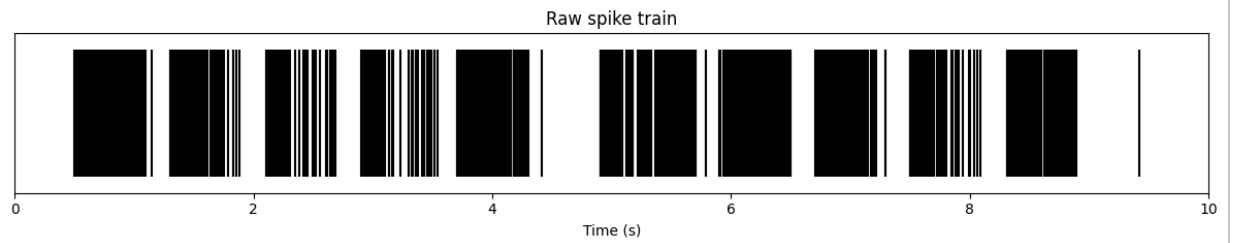
n_spikes = len(spike_times_s)
r_mean = n_spikes / T
print(f"Number of spikes: {n_spikes}")
print(f"Mean spike count-rate: {r_mean:.2f} spikes/s")
```

```
Number of spikes: 260
Mean spike count-rate: 65.00 spikes/s
```

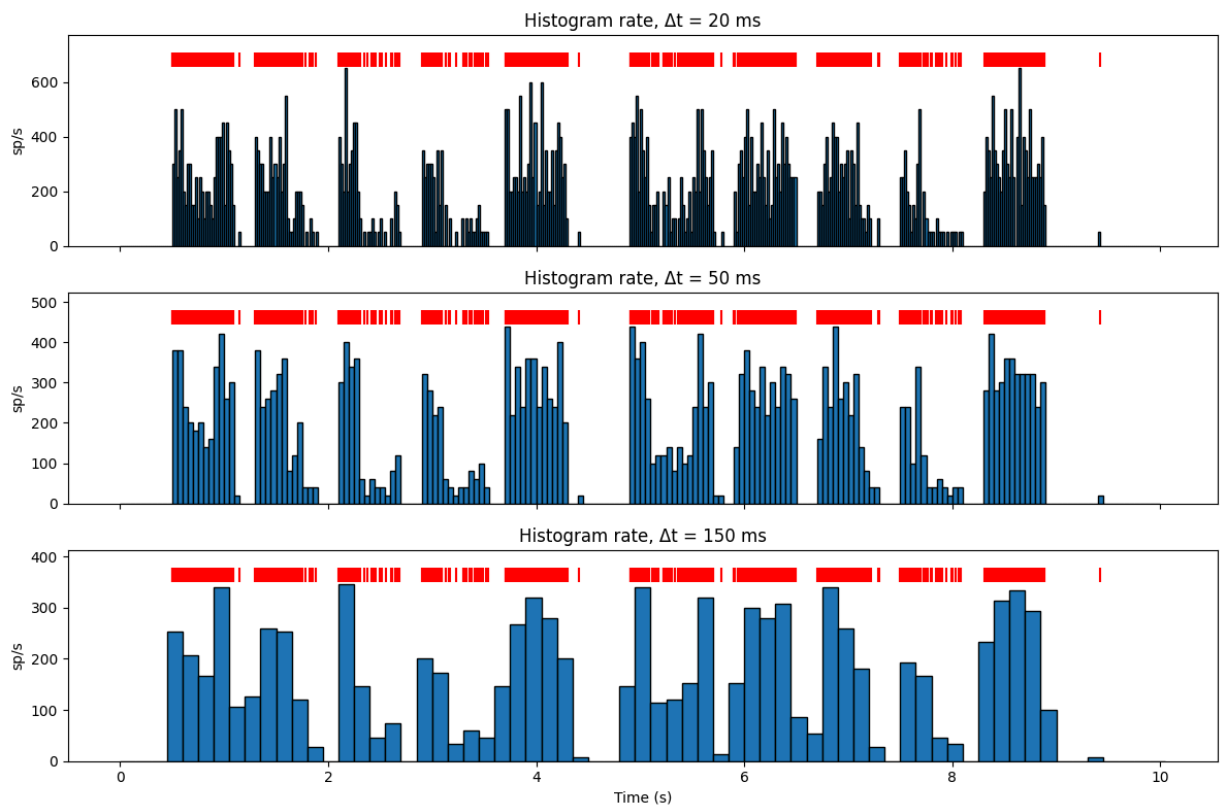
2. Extra: estimation of firing rate and trial averaging (3 extra points)

(a) (1 extra point) Estimate firing rates for the spike data in ExampleSpikeTimes2.dat to see if your code is flexible enough to handle it. Spike timings are in milliseconds, the trial length is $T = 10$ s, and the resolution is 0.1 ms.

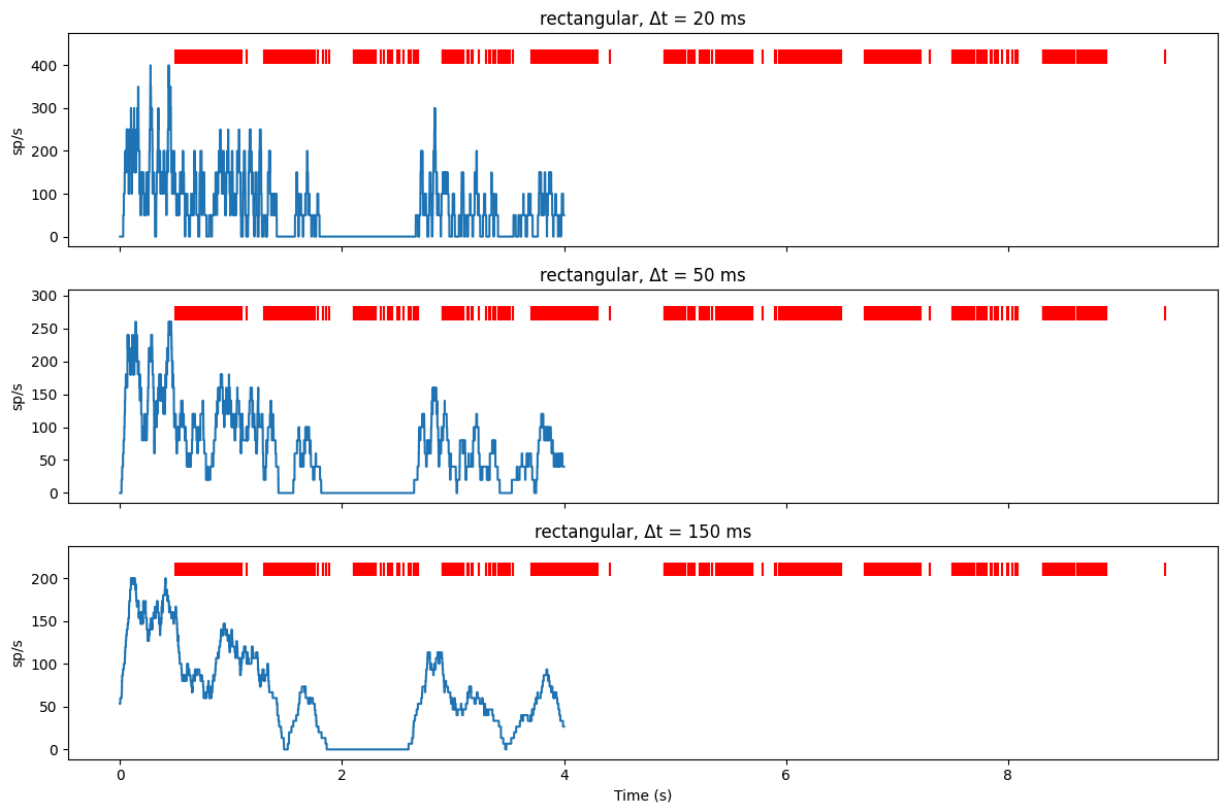
```
spike_times_s = load_spike_times("/content/ExampleSpikeTimes2.dat")
T = 10
dt = 0.0001
plot_spike_train(spike_times_s, T, dt)
```



```
bin_widths_ms = [20, 50, 150]
rate = plot_rate_histogram(spike_times_s, bin_widths_ms, T, dt)
```



```
plot_firing_rate_with_kernel('rectangular',[20, 50, 150], T, dt)
```



```
plot_firing_rate_with_kernel('gaussian',[10, 20, 50], T, dt)
```

gaussian, $\Delta t = 10$ ms